Air Quality Index

Prediction System

72-Hour AQI Forecasting · District Tharparkar, Pakistan

Shanker Lal

Data Science Intern

10Pearls · Data Science Internship Programme

72-hr	2 Models	0 Cost	Daily
Forecast Window	XGB + RF	Free Tier Infra	Auto Retraining

TECHNICAL REPORT

Executive Summary

Tharparkar sits at the edge of the Thar Desert and is home to one of Pakistan's largest active coal mining operations. That combination — industrial combustion layered on top of a naturally dusty, wind-swept landscape — creates air quality conditions that are both severe and genuinely difficult to anticipate.

When given the opportunity to work on a meaningful data science problem during my 10Pearls internship, building a forecasting system for this region felt like the right choice: technically challenging, and something that could actually help real people make safer daily decisions.

What I built is a self-sustaining pipeline that pulls meteorological and pollutant data, engineers temporal features, retrains two ensemble models every day, and serves 72-hour AQI forecasts through a public Streamlit dashboard. Once deployed, it runs itself — free-tier cloud infrastructure, zero monthly cost, zero manual intervention.

> This report is the full technical story: from first API call to a fully automated production pipeline running at zero operational cost.

Key Achievements

✓ 72-hr Forecast	Automatically refreshed AQI outlook served daily to the public dashboard.	**✓ Dual ML Models**	XGBoost + Random Forest retrained every day on live data — best RMSE wins.
✓ AWS S3 Registry	Model artifacts fully decoupled from Git; versioned object storage on S3.	**✓ MongoDB Atlas**	Real-time feature store (free-tier M0) for live inference data.
✓ Full CI/CD	End-to-end automation via GitHub Actions — no human needed to retrain.	**✓ Zero Secrets Leak**	AWS and MongoDB credentials injected via secrets vaults; zero hardcoding.
✓ Linear Git History	Clean, auditable commit log despite concurrent automated daily commits.	**✓ Zero Monthly Cost**	Free-tier infra across every layer: Streamlit, MongoDB, S3, GitHub Actions.

System Architecture

From the start, a one-off notebook wasn't what this project called for. If the forecast was going to be useful, it had to update itself. I structured the system around a pipeline of independent, schedulable stages — each responsible for exactly one thing and deployable independently.

End-to-End Pipeline

Stage	Component	Technology	Schedule
1 · Ingestion	Fetch met + pollutant data	Open-Meteo / OpenAQ APIs	Daily
2 · Processing	Parquet storage	Pandas, PyArrow	Daily
3 · Feature Eng.	Lag, rolling, wind vector feats	Scikit-Learn, Pandas	Daily
4 · Feature Store	Persist live features	MongoDB Atlas M0	Daily
5 · Training	XGBoost + Random Forest	XGBoost, Scikit-Learn	Daily
6 · Model Registry	Push versioned .joblib artifacts	AWS S3 + boto3	Daily
7 · Inference	72-hr AQI forecast served live	Streamlit Community Cloud	On boot

Technology Stack

Layer	Technology
Machine Learning	XGBoost, Random Forest, Scikit-Learn
Data Processing	Pandas, PyArrow (Parquet columnar storage)
Cloud Storage	AWS S3 — versioned model artifact registry
Feature Store	MongoDB Atlas M0 (free tier)
Frontend / Dashboard	Streamlit Community Cloud
CI/CD Automation	GitHub Actions — automated daily retraining workflow

Data Pipeline

Getting the data right took longer than expected. The gap between raw API output and genuinely predictive features is much wider than it appears — and closing that gap is where most of the real engineering time went.

Data Collection

Air quality and meteorological data for the Tharparkar region are pulled from two public APIs: Open-Meteo for weather variables and OpenAQ for pollutant concentrations. Neither requires authentication keys — a genuine advantage when configuring CI/CD runners, since it removes one class of credential to manage. Raw measurements are stored as Parquet files using PyArrow for columnar compression and strict type safety.

Category	Variables
Air Pollutants	PM2.5, PM10, CO, NO2, SO2, O3, Dust
Weather	Temperature, Humidity, Wind Speed, UV Index
Temporal	Timestamp (hourly granularity)

Feature Engineering

Raw pollutant readings tell you what air quality is right now — they say almost nothing about where it is heading. To give the model genuine temporal awareness, I engineered a set of derived features around each raw measurement.

Feature Type	Count	Description
Time-Based	10	Hour of day, day of week, month — linear and cyclical (sine/cosine encodings).
Lag Features	12	1-hour and 24-hour lags for PM2.5, PM10, AQI, temperature, and humidity.
Rolling Averages	6	3-hour and 24-hour moving averages for PM2.5, PM10, and AQI.
Derived / Interaction	3	Rate-of-change for AQI and PM2.5, plus a temperature-humidity interaction term.
Wind Vectors	2	Wind speed decomposed into u/v components for directional awareness.

Machine Learning Models

I chose XGBoost and Random Forest for reasons that went beyond benchmark scores. Both handle non-linear relationships well — which matters here because wind speed doesn't increase AQI linearly, and the interaction between temperature inversions and PM2.5 concentration is genuinely complex.

Model	Configuration	Strengths
XGBoost	200 estimators · max depth 6 · LR 0.05	Captures complex feature interactions; strong on structured tabular data with temporal patterns.
Random Forest	100 decision trees · max depth 10	Robust to outliers; generalises well across varying pollution regimes; reliable baseline.

After each daily retraining cycle, both models are evaluated on a held-out test set. Whichever achieves the lower RMSE automatically becomes active — preventing a silently degrading model from ever serving stale predictions.

9.47	12.24	13.37
+24 h RMSE	+48 h RMSE	+72 h RMSE

XGBoost production accuracy (RMSE) across forecast horizons — lower is better

Engineering Challenges & Solutions

The hardest part of this project wasn't the machine learning — it was making everything work together reliably across different cloud environments and automated schedules. Three problems pushed me well outside standard coursework into real production engineering territory.

Challenge 1 — Migrating from Version Control to a True Model Registry

My first instinct was to save trained .joblib files directly into the repository and let Streamlit load them from a relative path. Within a few retraining cycles, I realised every daily run was committing new binary files to Git, ballooning the repository size. Git doesn't diff binaries meaningfully, and it couples the inference layer tightly to source code. A bad model committed would immediately serve bad predictions.

The Solution:

I provisioned a dedicated AWS S3 bucket and restructured the pipeline so model artifacts never touch the repository. After each training run, model_training.py uses boto3 to push the .joblib files and fitted scaler to S3 with consistent, predictable object keys. The Streamlit app was rewritten as a lightweight S3 client — on boot it fetches the latest artifacts into in-memory buffers. Nothing lands on disk; binary files were evicted from version control entirely.

Challenge 2 — CI/CD Asynchrony and Git Synchronisation Conflicts

The GitHub Actions workflows commit data updates to the main branch on their own schedule. Every local push attempt was rejected with a 'fetch first' error. The naive fix — git pull then git push — creates a merge commit for every sync, filling the history with noise.

The Solution:

I switched to git pull --rebase as the standard workflow. Instead of merging histories, rebase replays local commits on top of whatever the remote has accumulated — preserving a linear, auditable history where every commit represents exactly one discrete action.

```
git stash # protect any unstaged local changes
git pull origin main --rebase # replay CI commits, re-apply my work on top
git stash pop # restore local changes cleanly
git push origin main # push with a clean, linear history
```

Challenge 3 — Secure Credential Injection Across Two Cloud Environments

Moving to AWS S3 and MongoDB Atlas meant both the training pipeline (GitHub Actions) and the inference pipeline (Streamlit) needed programmatic credentials. Hardcoding them in a public repository was off the table.

The Solution:

I treated each platform's native secrets mechanism as its own isolated vault. GitHub Actions stores AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, and MONGO_URI as Repository Secrets injected at execution time. Streamlit Cloud stores the same credentials in its Secrets configuration accessed via st.secrets. Both boto3 and pymongo pick up credentials through the standard environment variable chain — the application code contains zero hardcoded values.

Dashboard Features

The Streamlit dashboard is the public face of the project — where all the engineering work becomes something a resident or public health official can actually use.

Feature	Description
Real-Time AQI	Current AQI pulled directly from the database, displayed with its EPA category and colour coding.
72-Hour Forecast	Interactive line chart showing predicted AQI for the next three days.
Daily Summary Cards	At-a-glance cards for each forecast horizon (+24h, +48h, +72h).
Health Alert System	Automated text descriptions mapping numerical predictions to EPA health standards.

Automation & CI/CD

Once deployed, the system runs itself. There is no manual step to fetch data, trigger retraining, or push models — GitHub Actions handles all of it on a cron schedule, every single day at midnight UTC.

Workflow	Schedule	What It Does
Daily AQI Model Training	Daily — 00:00 UTC	Runs backfill.py to update historical data, trains both models, evaluates RMSE, pushes best artifacts to AWS S3, and commits Parquet data updates back to Git.

Strengths & Limitations

What Works Well

- Rolling-window and lag features give the model genuine temporal awareness, capturing trends that raw readings alone cannot.
- Daily retraining ensures the models continuously adapt as seasonal pollution patterns shift throughout the year.
- Dynamic model selection (lowest RMSE wins) prevents a silently degrading model from serving stale predictions to the public.
- The entire architecture relies on free-tier infrastructure — zero monthly operational cost across all layers.

Known Limitations

- Sudden pollution events — industrial accidents or severe dust storms — cannot be forecast without advance warning signals in the data.
- Accuracy degrades beyond the 72-hour horizon, which defines the practical boundary.
- The free-tier MongoDB Atlas M0 cluster imposes connection limits that will eventually require an upgrade as the historical dataset grows.

Project Structure

```
tharparkar-aqi/
.github/workflows/
daily_training.yml # Data update, retrain, S3 upload logic
backfill.py # API ingestion and historical data generation
preprocessing.py # Feature engineering, temporal lag creation, scaling
model_training.py # XGBoost + RF training and AWS boto3 client
app.py # Streamlit dashboard and S3 download client
requirements.txt # Python dependencies
README.md # Project documentation
```

Setup & Deployment

Step 1. Fork the repository and enable GitHub Actions in the repository settings.

Step 2. Provision an AWS S3 bucket and create an IAM user with s3:PutObject and s3:GetObject permissions.

Step 3. Create a MongoDB Atlas M0 cluster (free tier) and copy the connection URI.

Step 4. Add AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, and MONGO_URI to GitHub Repository Secrets.

Step 5. Deploy app.py to Streamlit Community Cloud and add the same credentials to Streamlit Secrets.

Step 6. Trigger the workflow manually once — data will backfill, models will train, and artifacts will upload to S3 automatically.

Live Links

Resource	URL
Live Dashboard	https://tharparkar-aqi.streamlit.app/
GitHub Repository	https://github.com/ShankerLal25150/aqi_prediction_Thar
CI/CD Actions	https://github.com/ShankerLal25150/aqi_prediction_Thar/actions

---

Built with curiosity, patience, and a genuine belief that better environmental data can improve people's lives.

— Shanker Lal · 10Pearls Data Science Internship